

# TREMAP

## Project Report & Comprehensive Architecture Documentation Trekking Management Application

---

|                          |                                                                                     |
|--------------------------|-------------------------------------------------------------------------------------|
| <b>Prepared By:</b>      | Abinash Mohanty                                                                     |
| <b>Email ID:</b>         | 24f2000448@ds.study.iitm.ac.in                                                      |
| <b>Date:</b>             | 1 July 2026                                                                         |
| <b>Technology Stack:</b> | Flask 3.x, Jinja2, HTML5, CSS3, Bootstrap 5 (Morph Theme), SQLite, Flask-SQLAlchemy |
| <b>Running Port:</b>     | Localhost   Port 8113                                                               |

## About Me

I am a student of the **IIT Madras BS degree** program, deeply interested and actively specializing in full-stack web development and scalable micro-framework systems design.

## AI / LLM Declaration

I have used AI solely for refinement, making the codebase look cleaner, and ensuring it is structurally well-organized. All the core concepts, logic flows, operational architectures, and schema structures are entirely my own design. AI assistance was utilized exclusively to streamline execution structure, ensure optimal environment efficiency, and provide polished direct user interface touches. **So finally all logic implementation, debugging and integration are done manually. At last, no use of AI/LLM to fulfill any kind of project requirements.**

## Video Report & Walkthrough

A complete screen share walkthrough presentation covering live system execution, multi-role workspace isolation, operational transitions, and technical validation can be viewed at the following link:

[https://drive.google.com/file/d/1e0Nwzaxc0rhTM0zCqpzQvpaPDl0iEIA1/view?usp=drive\\_link](https://drive.google.com/file/d/1e0Nwzaxc0rhTM0zCqpzQvpaPDl0iEIA1/view?usp=drive_link)

## 1. Problem Statement & Proposed Solution

**Problem Statement:** Modern adventure travel and scheduling require precise management of high-risk trekking itineraries, coordinator/guide validation, slot capacities, and booking transparent flows. Handling this ecosystem through manual lists or fragmented systems leads to booking conflicts, unverified staff accessing field assignments, oversell of slots, and general operational overhead.

**Proposed Solution:** TREMAP provides a centralized, modular micro-framework ecosystem built using role-based multi-tier controllers. Running on Flask 3.x with a programmatic SQLAlchemy object relationship configuration, the system isolates critical administration pathways. It features safe server-side password cryptography powered by the pbkdf2:sha256 hashing mechanism with an independent salt layer to secure client entries. Access control segregates users into three clean functional tiers:

- **Director Tier (Admin):** Superuser status with global permissions to launch journeys, initialize scheduling timelines, authorize or ban accounts, and approve pending coordinators.
- **Coordinator Tier (Staff):** Verified field personnel assigned explicitly to a journey who track and manage specific itinerary statuses.
- **Customer Tier (User/Trekker):** General customers equipped to query destinations seamlessly by location filters, execute atomic slot ticket purchases, and audit travel history.

## 2. System Architecture & Folder Structure

The platform strictly respects the Controller-based Model-View-Controller (MVC) separation layout pattern:

```

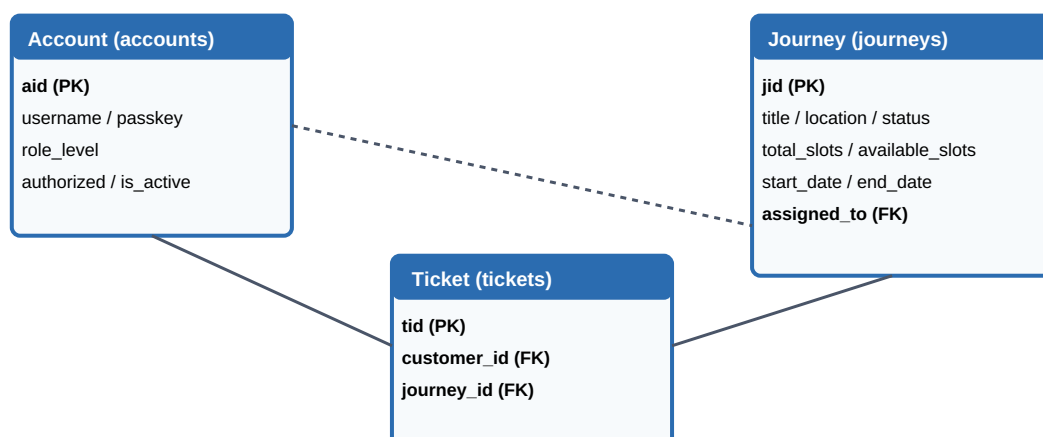
Trekking Management Application/
├─ app.py                ← Application initialization engine & Blueprint register.
├─ models/
│   └─ schemas.py        ← Declares data representations (Account, Journey, Ticket).
├─ controllers/
│   ├── auth_ctrl.py     ← Authentication workflow handling login/registration.
│   ├── director_ctrl.py ← Blueprint managing administrative capabilities.
│   ├── coordinator_ctrl.py ← Blueprint controlling staff dashboard views.
│   └─ customer_ctrl.py  ← Blueprint handling customer searches and bookings.
├─ templates/            ← Jinja2 front-end presentation context layers.
│   ├── base.html        ← Root layout leveraging the Bootstrap Morph theme.
│   ├── login.html / register.html
│   ├── director_dash.html
│   ├── coordinator_dash.html
│   └─ customer_dash.html
└─ TMA.sqlite            ← Programmatically generated SQLite storage database.

```

### 3. Database Models & Entity-Relationship Diagram

The persistence model defines three interconnected relational structures within `models/schemas.py`:

- **Account (Table: 'accounts')**: Models credentials and access authorizations. Distinct attributes include `role_level`, `authorized` (critical for pending guides), and `is_active` (handling user bans).
- **Journey (Table: 'journeys')**: Maps scheduled treks. Tracks inventory thresholds via `total_slots` and `available_slots`, linked to an Account via the `assigned_to` foreign key relation.
- **Ticket (Table: 'tickets')**: Operates as the transactional receipt linking a single customer's Account entity to a targeted Journey object.



## 4. Core Application Operational Flow

- **Authentication Boundary:** App runs on port `8113` via `app.py`. Users access `/auth/login` where credentials and blocklists are checked.
- **Registration Branching:** Registering as a 'customer' provides immediate login access. Registering a 'coordinator' locks the account in an unauthorized pool until explicitly unlocked by an administrator.
- **Director Administration:** The admin uses `/director/dashboard` to approve pending staff, apply security bans, and publish new journeys bound to designated coordinators.
- **Staff Manifest Management:** Approved guides check `/coordinator/dashboard` to monitor participants and transition journey states between Scheduled, Active, and Completed.
- **Customer Transaction Loop:** Trekkers search destinations on `/customer/dashboard` and issue an atomic ticket purchase via `/customer/buy\_ticket/<jid>`, programmatically decrementing available slot inventories.

## 5. Comprehensive Route Mapping Matrix

| Blueprint Module | Target URL Route Path      | HTTP Methods | Operational Integrity & Scope Actions                                                          |
|------------------|----------------------------|--------------|------------------------------------------------------------------------------------------------|
| root             | /                          | GET          | Intercepts landing queries and redirects cleanly into the login portal.                        |
| auth             | /auth/login                | GET/POST     | Validates password hashing, evaluates status tags, and serves corresponding dashboards.        |
| auth             | /auth/register             | GET/POST     | Handles enrollment workflows, auto-approving customers while putting staff on hold.            |
| auth             | /auth/logout               | GET          | Clears active session payloads and performs safe redirect teardowns.                           |
| director         | /director/dashboard        | GET          | Administrative console tracking comprehensive system accounts and global journeys.             |
| director         | /director/add_journey      | POST         | Validates tracking parameters, binds staff keys, and creates expedition packages.              |
| director         | /director/auth_coord/<aid> | POST         | Updates authorization status from false to true for the selected coordinator ID.               |
| director         | /director/ban_user/<aid>   | POST         | Sets active status to false, revoking system dashboard access for the targeted account.        |
| coordinator      | /coordinator/dashboard     | GET          | Isolated dashboard showing only the expeditions explicitly managed by the authenticated guide. |

| Blueprint Module | Target URL Route Path             | HTTP Methods | Operational Integrity & Scope Actions                                                                  |
|------------------|-----------------------------------|--------------|--------------------------------------------------------------------------------------------------------|
| coordinator      | /coordinator/update_journey/<jid> | POST         | Modifies active journey states and handles mid-journey slot inventory corrections.                     |
| customer         | /customer/dashboard               | GET          | Renders open expeditions, provides dynamic destination search filters, and catalogs purchased tickets. |
| customer         | /customer/buy_ticket/<jid>        | POST         | Processes atomic transactions: verifies slot capacity and creates a secure ticket record.              |

## 6. Core Business Logic & Architectural Constraints

- **Verification Enforcement:** Unapproved or banned accounts are blocked at the authentication gate, preventing deep route injection attempts.
- **Transactional Guardrails:** Customers cannot purchase tickets for itineraries where `available_slots` is 0 or the status is not marked explicitly as Scheduled.
- **Frontend Styling Blueprint:** Implemented via the Bootswatch Bootstrap 5 Morph framework, which delivers soft, modern, neumorphic interface states completely without client-side JavaScript dependencies for core platform mechanisms.

## 7. Summary

**TREMAP** successfully addresses trekking administration requirements by implementing a reliable, role-segregated architecture. Leveraging Python's Flask framework combined with programmatic SQLAlchemy SQLite hooks, the platform provides automated workflow enforcement. This design delivers a secure and responsive solution for travel administrators, field staff, and customers alike.